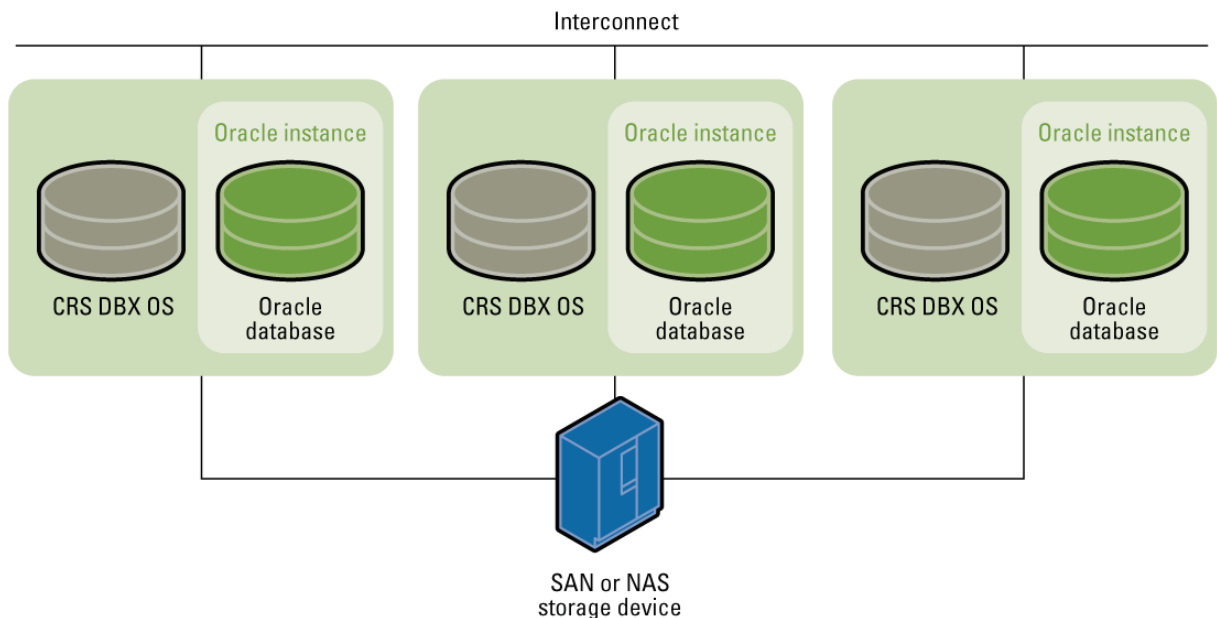


Unit - 4

Database cluster

Database clustering is the process of connecting more than one single database instance or server to your system. In most common database clusters, multiple database instances are usually managed by a single database server called the master. In the systems design world, implementing such a design may be necessary especially in large systems (web or mobile applications), as a single database server would not be capable of handling all of the customers' requests. To fix this issue, the utilization of multiple database servers that work in parallel will be introduced to the system.

It goes without saying that using such a technique comes with numerous benefits to our system such as handling more users and overcoming system failures. One of the main disadvantages of such implementation is the additional complexity introduced into the system. To handle additional complexity, multiple database servers should be managed by a higher-level server that monitors the flow of data throughout the system.

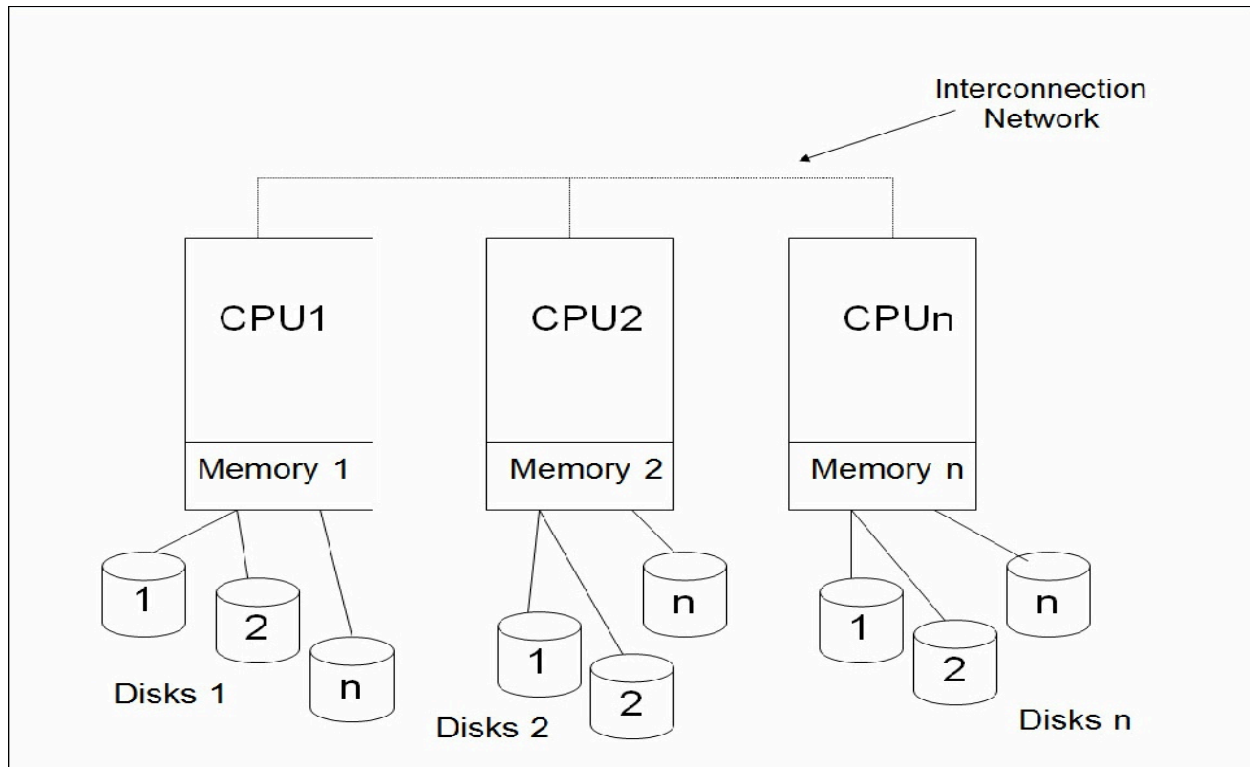


As shown in the above image, multiple database servers are connected together using a SAN device. **SAN** short for **Storage area network** is a computer network device that provides access to consolidated, block-level data storage. SANs are primarily used to access data storage devices, such as disk arrays and tape libraries from servers so that the devices appear to the operating system as direct-attached storage. While you still can build your own database cluster, recently, companies do provide third-party cloud database storage as a service for customers. Using such services customers can save costs on maintaining and monitoring their own database servers or clusters.

Database Cluster Architecture

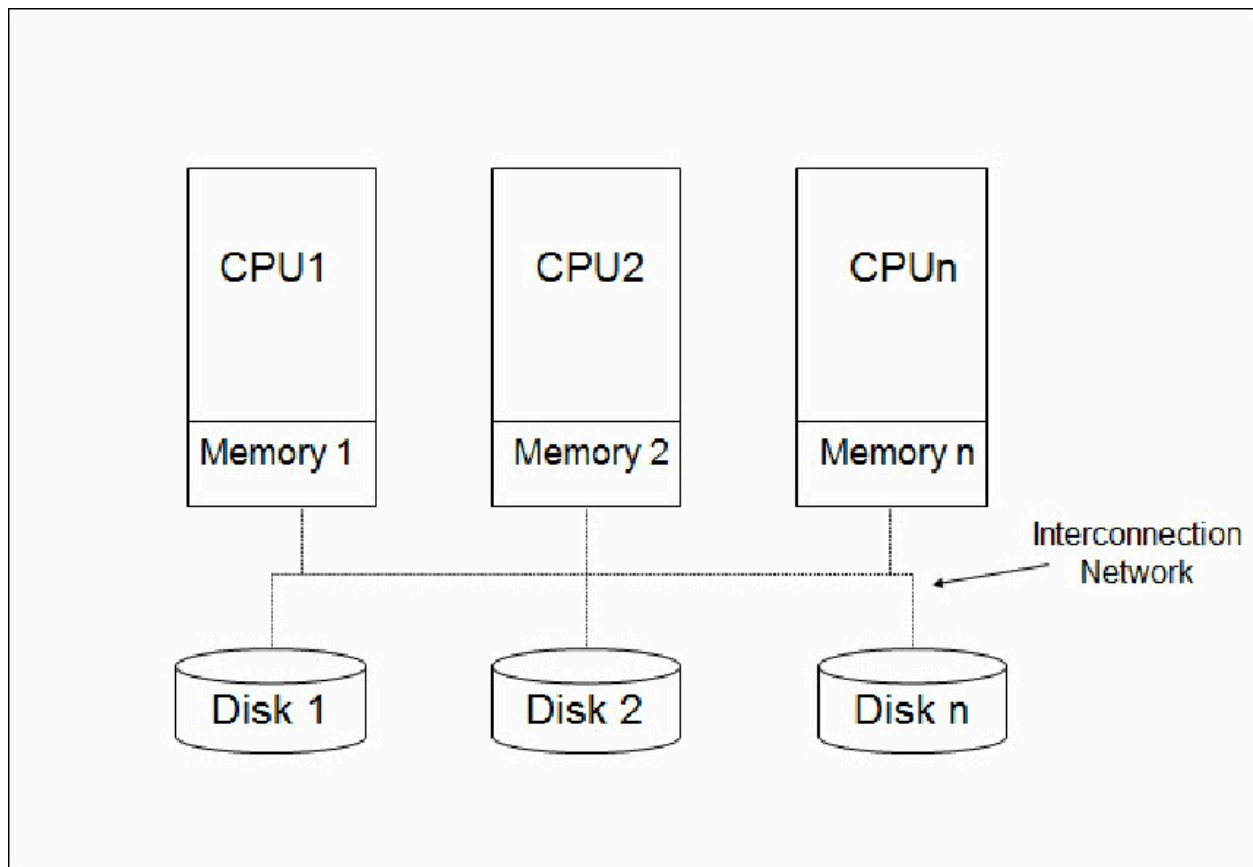
Shared-Nothing Architecture

To build a shared-nothing database architecture each database server must be independent of all other nodes. Meaning that each node has its own database server to store and access data from. In this type of architecture, no single database server is master. Meaning that there is no one central database node that monitors and controls the access of data in the system. Note that a shared-nothing architecture offers great horizontal scalability as no resources are being shared between either nodes or database servers.



Shared-Disk Architecture

On the other hand, we have the shared-disk architecture. In this architecture, all nodes(CPU) share access to all the database servers available, subsequently having access to all the system's data. Unlike the shared-nothing architecture, the interconnection network layer is between the CPU and the database servers allowing for multiple database servers' access. It is worth noting that a shared disk cluster does not offer much scalability when compared to the shared-nothing architecture, as if all nodes share access to the same data a controlling node is required to monitor the data flow in the system. The issue is that after exceeding a certain number of slave nodes, the master node would be unable to monitor and control all the slave nodes efficiently.



A shared disk architecture

Advantages of database clustering

Benefits of Database Clustering

There are several benefits to using database clustering in an organization. These include:

Improved Performance: Database clustering can improve the performance of a database system by distributing the workload across multiple nodes. This can reduce the load on individual servers and ensure that the system can handle a larger volume of requests.

High Availability: Database clustering can improve the availability of a database system by ensuring that data is replicated across multiple nodes. This means that if one node fails, the other nodes can continue to operate, ensuring that the database remains available to users.

Scalability: Database clustering can improve the scalability of a database system by allowing organizations to add additional nodes as needed. This means that organizations can easily scale their database system as their needs grow.

Fault Tolerance: Database clustering can improve the fault tolerance of a database system by ensuring that data is replicated across multiple nodes. This means that if one node fails, the other nodes can continue to operate, ensuring that data is not lost.

Cost Savings: Database clustering can provide cost savings by allowing organizations to use commodity hardware instead of expensive, high-end servers. Additionally, clustering can reduce the need for specialized IT personnel, as the system can be managed using off-the-shelf tools.

Indexing in DBMS

- Indexing is used to optimize the performance of a database by minimizing the number of disk accesses required when a query is processed.
- The index is a type of data structure. It is used to locate and access the data in a database table quickly.

Index structure:

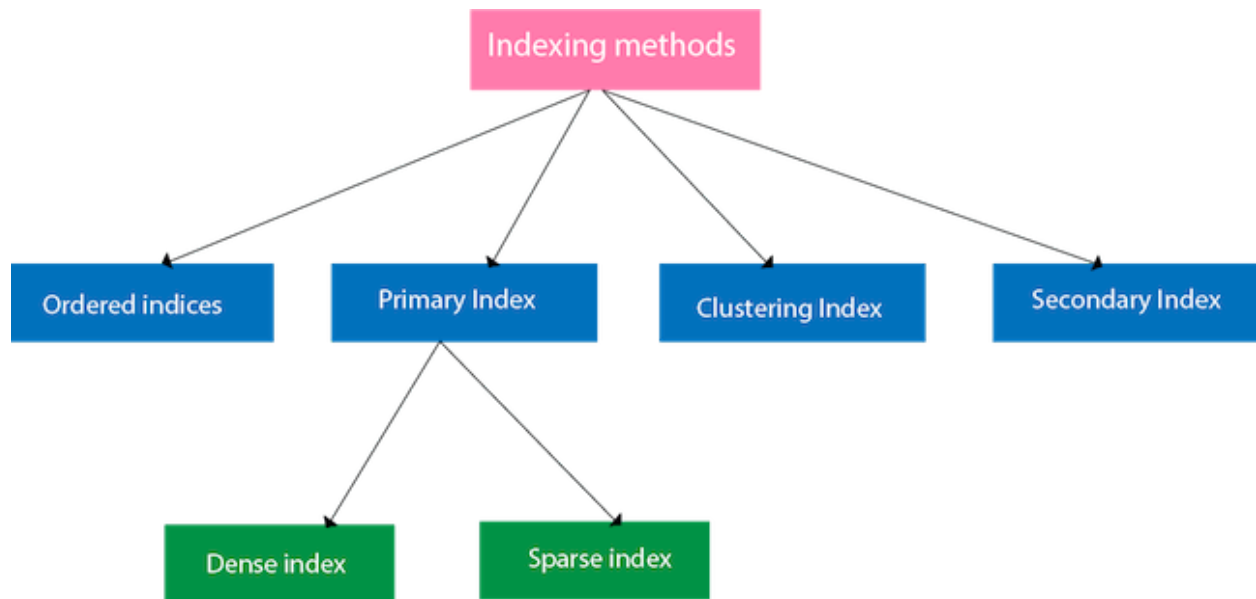
Indexes can be created using some database columns.

Search key	Data Reference
------------	-------------------

Fig: Structure of Index

- The first column of the database is the search key that contains a copy of the primary key or candidate key of the table. The values of the primary key are stored in sorted order so that the corresponding data can be accessed easily.
- The second column of the database is the data reference. It contains a set of pointers holding the address of the disk block where the value of the particular key can be found.

Indexing Methods



Ordered indices

The indices are usually sorted to make searching faster. The indices which are sorted are known as ordered indices.

Example: Suppose we have an employee table with thousands of record and each of which is 10 bytes long. If their IDs start with 1, 2, 3....and so on and we have to search student with ID-543.

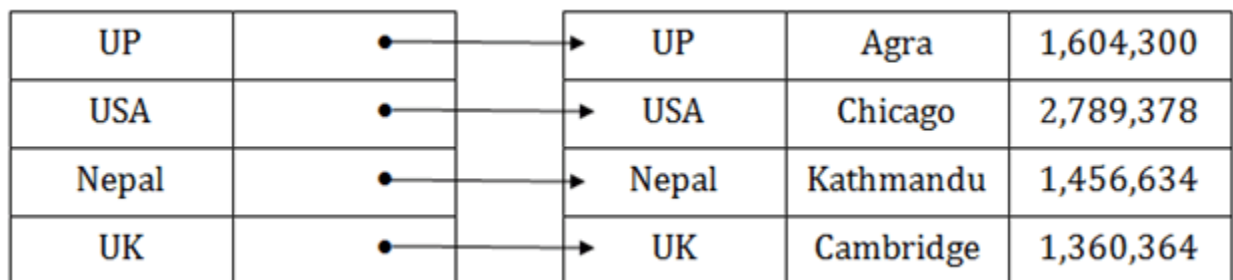
- In the case of a database with no index, we have to search the disk block from starting till it reaches 543. The DBMS will read the record after reading $543 \times 10 = 5430$ bytes.
- In the case of an index, we will search using indexes and the DBMS will read the record after reading $542 \times 2 = 1084$ bytes which are very less compared to the previous case.

Primary Index

- If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.
- As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.
- The primary index can be classified into two types: Dense index and Sparse index.

Dense index

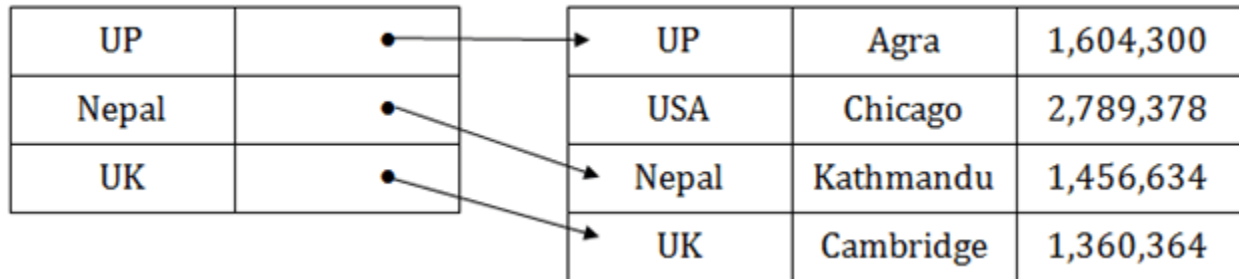
- The dense index contains an index record for every search key value in the data file. It makes searching faster.
- In this, the number of records in the index table is same as the number of records in the main table.
- It needs more space to store index record itself. The index records have the search key and a pointer to the actual record on the disk.



Sparse index

- In the data file, index record appears only for a few items. Each item points to a block.

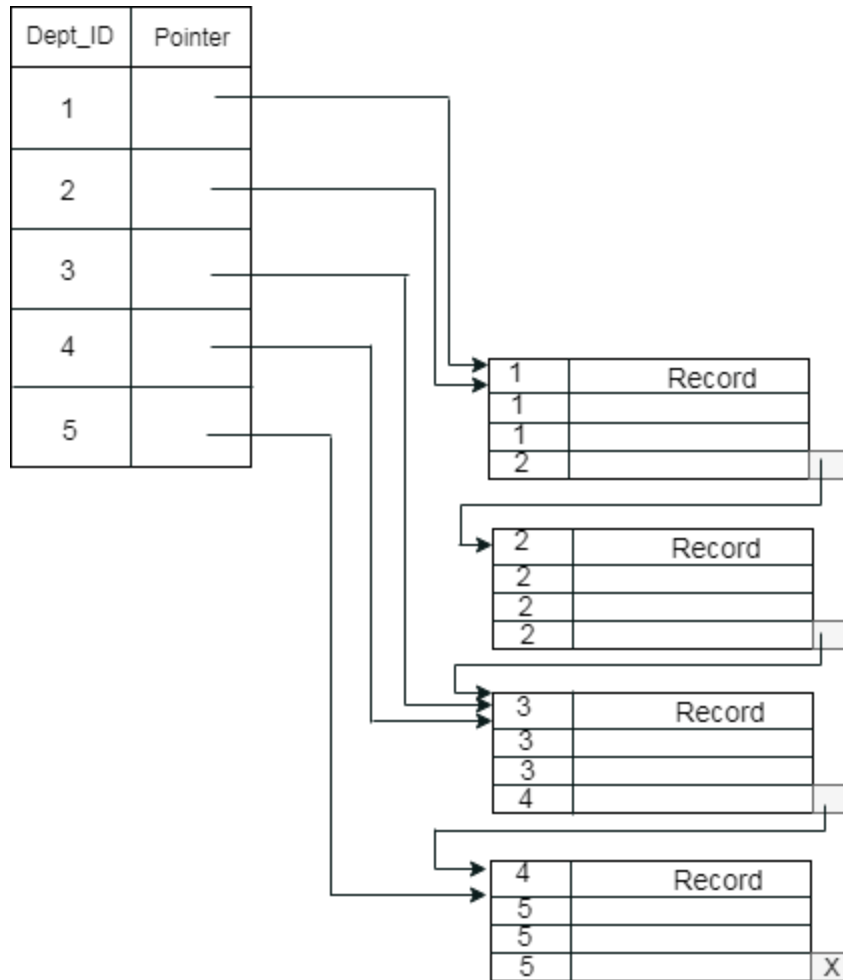
- In this, instead of pointing to each record in the main table, the index points to the records in the main table in a gap.



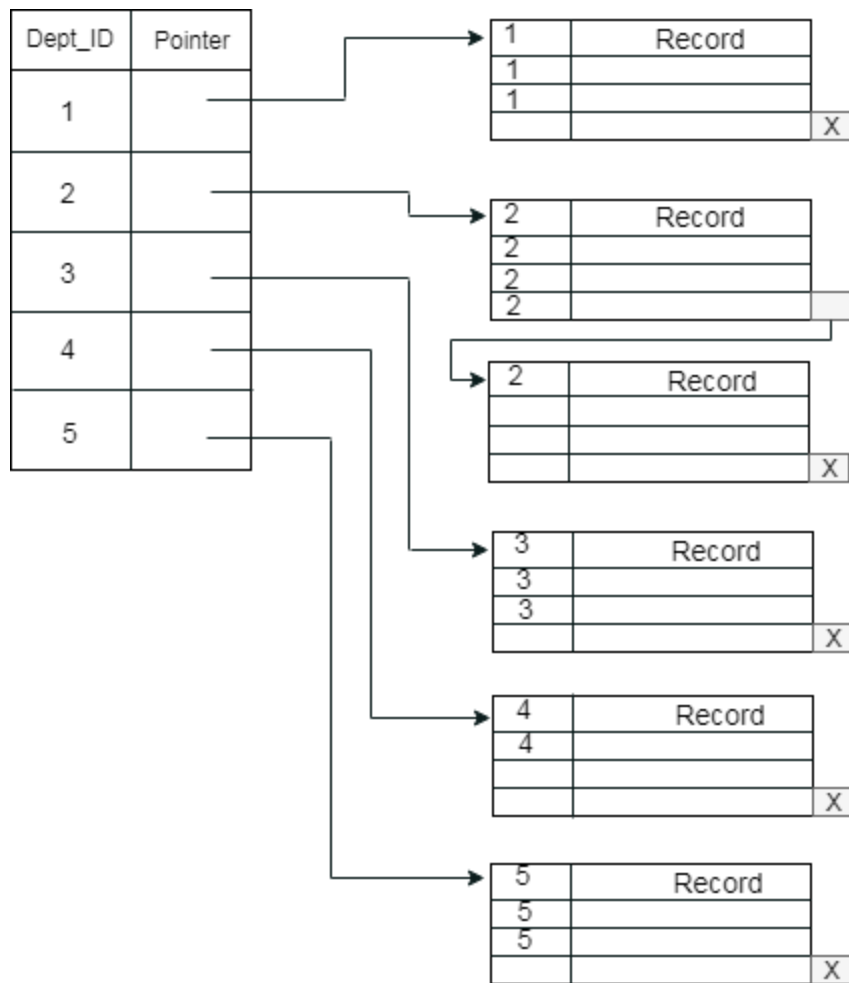
Clustering Index

- A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record.
- In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index.
- The records which have similar characteristics are grouped, and indexes are created for these group.

Example: suppose a company contains several employees in each department. Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole. Here Dept_Id is a non-unique key.



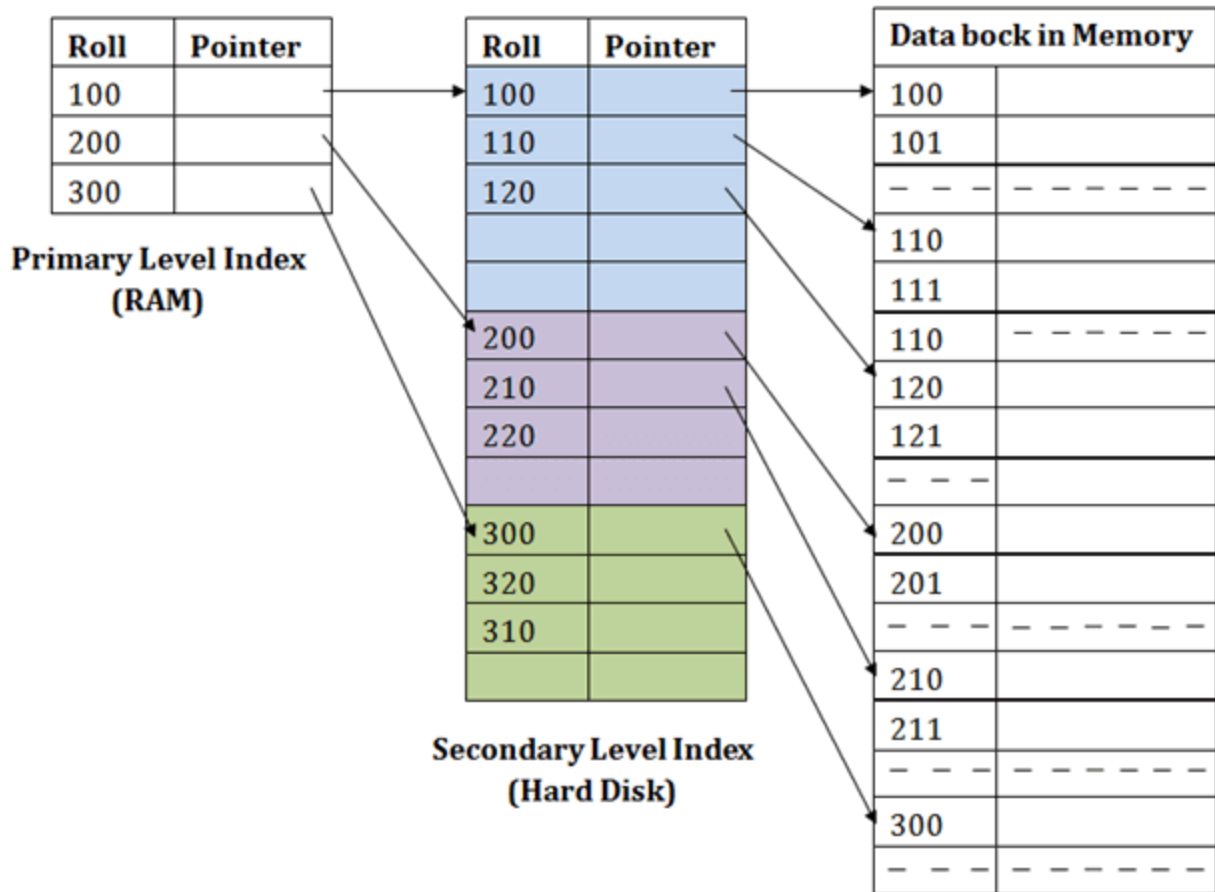
The previous schema is little confusing because one disk block is shared by records which belong to the different cluster. If we use separate disk block for separate clusters, then it is called better technique.



Secondary Index

In the sparse indexing, as the size of the table grows, the size of mapping also grows. These mappings are usually kept in the primary memory so that address fetch should be faster. Then the secondary memory searches the actual data based on the address got from mapping. If the mapping size grows then fetching the address itself becomes slower. In this case, the sparse index will not be efficient. To overcome this problem, secondary indexing is introduced.

In secondary indexing, to reduce the size of mapping, another level of indexing is introduced. In this method, the huge range for the columns is selected initially so that the mapping size of the first level becomes small. Then each range is further divided into smaller ranges. The mapping of the first level is stored in the primary memory, so that address fetch is faster. The mapping of the second level and actual data are stored in the secondary memory (hard disk).



For example:

- If you want to find the record of roll 111 in the diagram, then it will search the highest entry which is smaller than or equal to 111 in the first level index. It will get 100 at this level.
- Then in the second index level, again it does $\max(111) \leq 111$ and gets 110. Now using the address 110, it goes to the data block and starts searching each record till it gets 111.
- This is how a search is performed in this method. Inserting, updating or deleting is also done in the same manner.

Guidelines for index Selection

Primary Index Selection Criteria

The following table summarizes the guidelines for selecting columns to be used as primary indexes.

Guideline	Comments
Select columns that are most frequently used to access rows.	Restrict selection to columns that are either unique or highly singular.
Select columns that are most frequently used in equality predicate conditions.	Equality conditions permit the system to hash directly to the row having the conditional value. When the primary index is unique, the response is never more than one row. Inequality conditions require additional processing.
Select columns that distribute rows evenly across the AMPs.	Distinct values distribute evenly across all AMPs in the configuration. This maximizes parallel processing. Rows having duplicate NUPI values hash to the same AMP and often are stored in the same data block. This is good when rows are only moderately nonunique. Rows having NUPI columns that are highly nonunique distribute unevenly, use multiple data blocks, and incur multiple I/Os. Extremely nonunique primary index values can skew space usage so markedly that the system returns a message indicating that the database is full even when it is not.

	This occurs when an AMP exceeds the maximum bytes threshold for a user or database calculated by dividing the PERMANENT = n BYTES specification by the number of AMPs in the configuration, causing the system to incorrectly perceive the database to be “full.”
Select columns that are not volatile.	Volatile columns force frequent row redistribution.
Select columns having very many more distinct values than the number of AMPs in the configuration.	If this guideline is not followed, row distribution skews heavily, not only wasting disk space, but also devastating system performance. This rule is particularly important for large tables.
Do not select columns defined with Period, ARRAY, VARRAY, Geospatial, JSON, XML, BLOB, CLOB, XML-based UDT, BLOB-based UDT, or CLOB-based UDT data types.	You cannot specify columns that have BLOB, CLOB, BLOB-based UDT, CLOB-based UDT, XML-based UDT, Period, ARRAY, VARRAY, Geospatial, or JSON data types in a primary index definition. If you attempt to do so, the CREATE request aborts. You can, however, specify Period data type columns in the partitioning expression of a partitioned table.

Do not select aggregated columns of a join index.	When defining the primary index for a join index, you cannot specify any aggregated columns. If you attempt to do so, the CREATE JOIN INDEX request aborts.
---	---

There are a few basic rules to keep in mind when choosing indexes for a database. A good index should have these three properties:

1. Usefulness: Speed up the execution of some queries (or enforce a constraint)
2. Clustering: Keep records that are likely to be accessed together near each other
3. Scattering: Keep records that are unlikely to be accessed together far apart

Table Size

It is not recommended to create indexes on small tables, as it takes the SQL Server Engine less time scanning the underlying table than traversing the index when searching for a specific data. In this case, the index will not be used and still affect the data modification performance, as it will be always adjusted when modifying the underlying table's data.

Table Columns

In addition to database workload characteristics, the characteristics of the table columns that are used in the submitted queries should be also considered when designing an index. For instance, the columns with exact numeric data types, such as INT and BIGINT data types and that are UNIQUE and NOT NULL are considered optimal columns to participate in the index key.

Columns Order and Sorting

It is recommended to create the indexes on columns that are used in the query predicates and join conditions in the correct order that is specified in the predicate. In this way, the goal is keeping the index key short, without including the rarely used columns, in order to minimize the index complexity, storage and maintenance overheads.

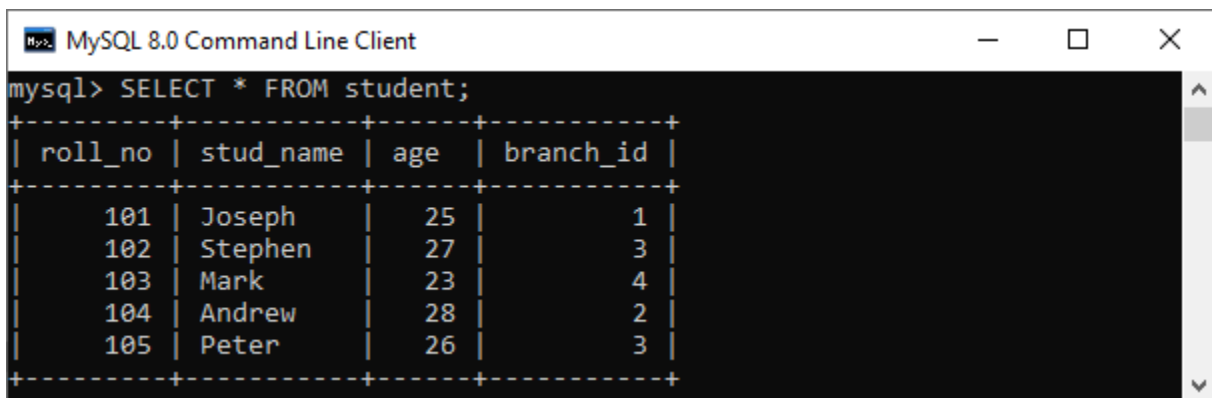
De-normalization

When we normalize tables, we break them into multiple smaller tables. So when we want to retrieve data from multiple tables, we need to perform some kind of join operation on them. In that case, we use the denormalization technique that eliminates the drawback of normalization.

Denormalization is a technique used by database administrators to optimize the efficiency of their database infrastructure. This method allows us to add redundant data into a normalized database to alleviate issues with database queries that merge data from several tables into a single table. The denormalization concept is based on the definition of normalization that is defined as arranging a database into tables correctly for a particular purpose.

NOTE: Denormalization does not indicate not doing normalization. It is an optimization strategy that is used after normalization has been achieved.

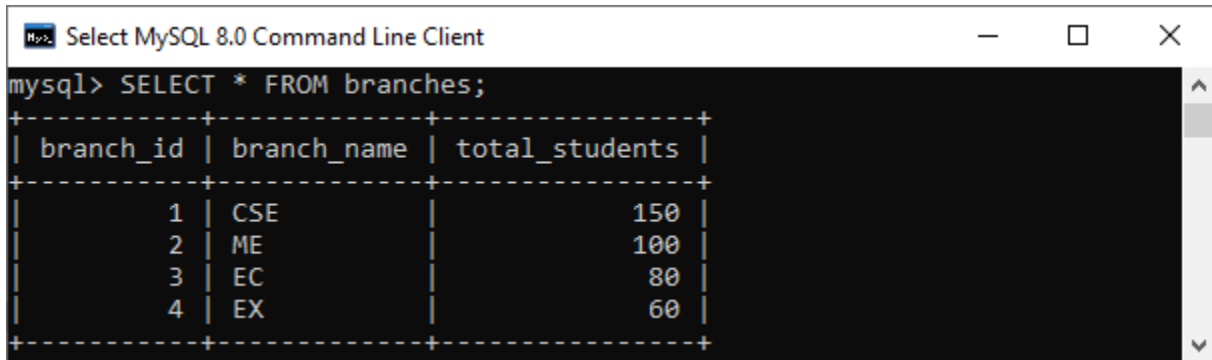
For Example, We have two table students and branch after performing normalization. The student table has the attributes roll_no, stud-name, age, and branch_id.



```
mysql> SELECT * FROM student;
```

roll_no	stud_name	age	branch_id
101	Joseph	25	1
102	Stephen	27	3
103	Mark	23	4
104	Andrew	28	2
105	Peter	26	3

Additionally, the branch table is related to the student table with branch_id as the student table's foreign key.



The screenshot shows a terminal window titled "Select MySQL 8.0 Command Line Client". The prompt is "mysql>". The command entered is "SELECT * FROM branches;". The output is a table with three columns: "branch_id", "branch_name", and "total_students". The data rows are: (1, CSE, 150), (2, ME, 100), (3, EC, 80), and (4, EX, 60).

branch_id	branch_name	total_students
1	CSE	150
2	ME	100
3	EC	80
4	EX	60

A JOIN operation between these two tables is needed when we need to retrieve all student names as well as the branch name. Suppose we want to change the student name only, then it is great if the table is small. The issue here is that if the tables are big, joins on tables can take an excessively long time.

In this case, we'll update the database with denormalization, redundancy, and extra effort to maximize the efficiency benefits of fewer joins. Therefore, we can add the branch name's data from the Branch table to the student table and optimizing the database.

Advantages of Denormalization

he following are the advantages of denormalization:

1. Enhance Query Performance

Fetching queries in a normalized database generally requires joining a large number of tables, but we already know that the more joins, the slower the query. To overcome this,

we can add redundancy to a database by copying values between parent and child tables, minimizing the number of joins needed for a query.

2. Make database more convenient to manage

A normalized database is not required calculated values for applications. Calculating these values on-the-fly will take a longer time, slowing down the execution of the query. Thus, in denormalization, fetching queries can be simpler because we need to look at fewer tables.

3. Facilitate and accelerate reporting

Suppose you need certain statistics very frequently. It requires a long time to create them from live data and slows down the entire system. Suppose you want to monitor client revenues over a certain year for any or all clients. Generating such reports from live data will require "searching" throughout the entire database, significantly slowing it down.

Disadvantages of Denormalization

The following are the disadvantages of denormalization:

- It takes large storage due to data redundancy.
- It makes it expensive to updates and inserts data in a table.
- It makes update and inserts code harder to write.
- Since data can be modified in several ways, it makes data inconsistent. Hence, we'll need to update every piece of duplicate data. It's also used to measure values and produce reports. We can do this by using triggers, transactions, and/or procedures for all operations that must be performed together.

How is denormalization different from normalization?

The denormalization is different from normalization in the following manner:

- Denormalization is a technique used to merge data from multiple tables into a single table that can be queried quickly. Normalization, on the other hand, is used to delete redundant data from a database and replace it with non-redundant and reliable data.
- Denormalization is used when joins are costly, and queries are run regularly on the tables. Normalization, on the other hand, is typically used when a large number of insert/update/delete operations are performed, and joins between those tables are not expensive.

Database Tuning

Database Tuning in SQL is a set of activities performed to optimize a database and prevents it from becoming a bottleneck.

There are various techniques with which you can configure the optimal performance of a particular database. Database tuning overlaps with query tuning; so, good indexing and avoiding improper queries help in increasing the database efficiency. In addition, increasing storage, updating to latest database versions and investing in a more powerful CPU (if needed) are also some of the general techniques.

Database Tuning Techniques

We can implement the following techniques to optimize the performance of a database

–

Database Normalization

Normalization is the process of removing of duplicate data from a database. We can normalize a database by breaking down larger tables into smaller related tables. This increases the performance of database as it requires less time to retrieve data from small tables instead of one large table.

Proper Indexes

In SQL, indexes are the pointers (memory address) to the location of specific data in database. We use indexes in our database to reduce query time, as the database engine can jump to the location of a specific record using its index instead of scanning the entire database.

Avoid Improper Queries

Choosing the correct query to retrieve data efficiently also improves the performance of a database. For example, choosing to retrieve an entire table when we only need the data in a single column will unnecessarily increase query time. So, query the database wisely.

Let us discuss some of the common improper queries made and how to rectify them to optimize the database performance.

1. Use SELECT fields instead of SELECT (*)

In large databases, we should always retrieve only the required columns from the database instead of retrieving all the columns, even when they are not needed. We can easily do this by specifying the column names in the SELECT statement instead of using the SELECT (*) statement.

Example

Assume we have created a table with name CUSTOMERS in MySQL database using CREATE TABLE statement as shown below –

```
CREATE TABLE CUSTOMERS (
```

```
  ID INT NOT NULL,
```

```
  NAME VARCHAR (20) NOT NULL,
```

```
  AGE INT NOT NULL,
```

```
  ADDRESS CHAR (25),
```

```
  SALARY DECIMAL (18, 2),
```

```
  PRIMARY KEY (ID)
```

```
);
```

Following query inserts values into this table using the INSERT statement –

```
INSERT INTO CUSTOMERS VALUES
```

```
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
```

```
(2, 'Khilan', 25, 'Delhi', 1500.00 ),
```

```
(3, 'Kaushik', 23, 'Kota', 2000.00 ),
```

```
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),
```

```
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),
```

```
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),
```

```
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

Let us say we only want the data in ID, NAME and SALARY columns of the CUSTOMERS table. So, we should only specify those three columns in our SELECT statement as shown below –

```
SELECT ID, NAME, SALARY FROM CUSTOMERS;
```

Output

The output obtained is as shown below –

ID	NAME	SALARY
1	Ramesh	2000.00
2	Khilan	1500.00
3	Kaushik	2000.00
4	Chaitali	6500.00
5	Hardik	8500.00
6	Komal	4500.00
7	Muffy	10000.00

2. Use Wildcards

Wildcards (%) are characters that we use to search for data based on patterns. These wildcards paired with indexes only improves performance because the database can quickly find the data that matches the pattern.

Example

If we want to retrieve the names of all the customers starting with K from the CUSTOMERS table, then, the following query will provide the quickest result –

```
SELECT ID, NAME FROM CUSTOMERS WHERE NAME LIKE 'K%';
```

Output

Following is the output of the above query –

ID	NAME
2	Khilan
3	Kaushik
6	Komal

3. Use Explicit Join

SQL JOINS are used to combine two tables based on a common column. There are two ways of creating a JOIN implicit join and explicit join. Explicit Join notation use the JOIN keyword with the ON clause to join two tables while the implicit join notation does not use the JOIN keyword and works with the WHERE clause.

Performance wise, they are both on the same level. However, in more complicated cases, the implicit join notation might produce completely different results than intended. Therefore, Explicit Joining is preferred.

4. Avoid using SELECT DISTINCT

The DISTINCT operator in SQL is used to retrieve unique records from the database. And on a properly designed database table with unique indexes, we rarely use it.

But, if we still have to use it on a table, using the GROUP BY clause instead of the DISTINCT keyword shows a better query performance (at least in some databases).

5. Avoid using Multiple OR

The OR operator is used to combine multiple conditions when filtering a database. Whenever we use OR in a filter condition each statement is processed separately. This degrades database performance as the entire table must be scanned multiple times to retrieve the data that matches the filter condition.

Instead, we can use a more optimized solution; by breaking the different OR conditions into separate queries, which can be processed parallelly by the database. Then, the results from these queries can be combined using UNION.

Example

For example, let us say we have a requirement of getting the details of all the customers whose age is greater than 25 or whose salary is greater than 2,000. The optimized query would be as show below –

```
SELECT ID, NAME FROM CUSTOMERS WHERE AGE > 25
```

```
UNION
```

```
SELECT ID, NAME FROM CUSTOMERS WHERE SALARY > 2000;
```

Output

After executing the above code, we get the following output –

ID	NAME
1	Ramesh
5	Hardik
4	Chaitali
6	Komal
7	Muffy

6. Use WHERE instead of HAVING

The WHERE and HAVING clause are both used to filter data in SQL. However, WHERE clause is more efficient than HAVING. With WHERE clause, only the records that match the condition are retrieved. But with HAVING clause, it first retrieves all the records and then filters them based on a condition. Therefore, the WHERE clause is preferable.

Functionality and Features

Database Performance Tuning involves various techniques and methodologies aimed at improving database performance, such as:

Query optimization: Rewriting SQL queries for better execution plans

Index management: Creating, modifying, and deleting indexes to optimize data access

Resource allocation: Assigning memory, CPU, and disk resources for optimal performance

Database design: Designing database schemas that allow efficient data processing

Data partitioning: Dividing large tables into smaller, more manageable pieces for improved query performance

Caching: Storing frequently accessed data in memory for faster retrieval

Built-In Tuning Tools

Some databases provide built-in tuning tools to monitor the database performance. For instance, the Oracle database provides the following tuning tools –

EXPLAIN – In SQL, the EXPLAIN command give us the order in which a query is executed along with the estimated cost of each step. We can use this to find the query the least cost to optimize the database.

tkprof – tkprof is a command that gives us various statistics, such as CPU and I/O usage of a query. By using these statistics, we can tune our queries to reduce CPU and I/O utilization to increase the efficiency of our database.

Database Security

Database security is the practice of protecting sensitive data and information stored in a database from unauthorized access, misuse, or destruction. It involves a range of techniques and strategies designed to ensure that only authorized individuals or entities can access and use the data stored in the database.

Database security is important for several reasons, as follows:

- Confidentiality: Safeguarding confidential data from unauthorized access or disclosure
- Integrity: Guaranteeing the integrity of data by preventing any unauthorized alterations or corruption
- Availability: Ensuring that data is available to authorized users when needed
- Compliance: Meeting regulatory and legal requirements related to data security and privacy

Types of Database Security

Numerous techniques are employed to safeguard databases against diverse threats, such as the following:



- **Physical Security:**

Physical security safeguards the database against unauthorized access or theft through tangible means. Measures such as securing the server room, implementing access controls for the data center, and employing security cameras and alarms are integral components of physical security.

The significance of physical security lies in its ability to safeguard the database hardware from physical harm or theft, ensuring its integrity and availability. Additionally, physical security plays a vital role in preventing unauthorized individuals from gaining access to the database servers or storage devices, thereby fortifying overall data security.

- **Network Security:**

Network security pertains to safeguarding the database against unauthorized network access. Network security measures include using firewalls, intrusion detection systems, and encryption.

Network security is important as it ensures that data is transmitted securely over the network. Unauthorized individuals cannot intercept or modify the

data in transit. Network security also helps protect the database from external attacks such as hacking and malware.

- Access Control:

Access control serves as a security methodology that limits database access exclusively to authorized users. It encompasses various measures such as authentication, authorization, and accounting, ensuring that only individuals with proper credentials can interact with the database.

Authentication ensures that only authorized individuals can access the database by verifying their identity using usernames and passwords.

Authorization controls what actions each user can perform on the database based on their role or privileges. Accounting ensures that all database activities are logged and audited for accountability and compliance.

Access control is critical as it ensures that only authorized individuals can access the database and perform specific actions on the data. Access control also helps prevent unauthorized data modifications or deletions.

- Data Encryption:

Data encryption is a technique used to protect data stored in a database from unauthorized access by encrypting it using encryption algorithms.

Encryption ensures that even if an unauthorized individual gains access to the data, they cannot read or use it.

Data encryption is important as it ensures that sensitive data is protected even if it falls into the wrong hands. Encryption also helps prevent data breaches and unauthorized access to the database.

- **Auditing and Logging:**

Auditing and logging serve as vital methods for overseeing and tracing all actions executed on the database, aiming to identify and prevent security breaches. These techniques encompass the comprehensive recording of various database activities, including user logins, data modifications, and system events.

Auditing and logging are critical as they provide a record of all activities performed on the database. This can be used to detect and prevent security breaches. Auditing and logging also help meet regulatory and compliance requirements related to data security and privacy.

Why Database Security is Important?

- **Intellectual property that is compromised:** Our intellectual property--trade secrets, inventions, or proprietary methods -- could be vital for our ability to maintain an advantage in our industry. If our intellectual property has been stolen or disclosed and our competitive advantage is lost, it could be difficult to keep or recover.
- **The damage to our brand's reputation:** Customers or partners may not want to purchase goods or services from us (or deal with our business) If they do not feel they can trust our company to protect their data or their own.
- **The concept of business continuity (or lack of it):** Some businesses cannot continue to function until a breach has been resolved.
- **Penalties or fines to be paid for not complying:** The cost of not complying with international regulations like the Sarbanes-Oxley Act (SAO) or Payment Card Industry Data Security Standard (PCI DSS) specific to industry regulations on

data privacy, like HIPAA or regional privacy laws like the European Union's General Data Protection Regulation (GDPR) could be a major problem with fines in worst cases in excess of many million dollars for each violation.

- **Costs for repairing breaches and notifying consumers about them:** Alongside notifying customers of a breach, the company that has been breached is required to cover the investigation and forensic services such as crisis management, triage repairs to the affected systems, and much more.

Common Threats and Challenges

Numerous software configurations that are not correct, weaknesses, or patterns of carelessness or abuse can lead to a breach of security. Here are some of the most prevalent kinds of reasons for security attacks and the reasons.

Insider Dangers

An insider threat can be an attack on security from any three sources having an access privilege to the database.

- A malicious insider who wants to cause harm
- An insider who is negligent and makes mistakes that expose the database to attack. vulnerable to attacks
- An infiltrator is an outsider who acquires credentials by using a method like phishing or accessing the database of credential information in the database itself.

Insider dangers are among the most frequent sources of security breaches to databases. They often occur as a consequence of the inability of employees to have access to privileged user credentials.

Human Error

The unintentional mistakes, weak passwords or sharing passwords, and other negligent or uninformed behaviours of users remain the root causes of almost half (49 percent) of all data security breaches.

Database Software Vulnerabilities can be Exploited

Hackers earn their money by identifying and exploiting vulnerabilities in software such as databases management software. The major database software companies and open-source databases management platforms release regular security patches to fix these weaknesses. However, failing to implement the patches on time could increase the risk of being hacked.

SQL/NoSQL Injection Attacks

A specific threat to databases is the infusing of untrue SQL as well as other non-SQL string attacks in queries for databases delivered by web-based apps and HTTP headers. Companies that do not follow the safe coding practices for web applications and conduct regular vulnerability tests are susceptible to attacks using these.

Buffer Overflow is a way to Exploit Buffers

Buffer overflow happens when a program seeks to copy more data into the memory block with a certain length than it can accommodate. The attackers may make use of the extra data, which is stored in adjacent memory addresses, to establish a basis for they can begin attacks.

DDoS (DoS/DDoS) Attacks

In a denial-of-service (DoS) attack in which the attacker overwhelms the targeted server – in this case, the database server with such a large volume of requests that the server is unable to meet no longer legitimate requests made by actual users. In most cases, the server is unstable or even fails to function.

Malware

Malware is software designed to exploit vulnerabilities or cause harm to databases. Malware can be accessed via any device that connects to the databases network.

Attacks on Backups

Companies that do not protect backup data using the same rigorous controls employed to protect databases themselves are at risk of cyberattacks on backups.

The following factors amplify the threats:

- **Data volumes are growing:** Data capture, storage, and processing continue to increase exponentially in almost all organizations. Any tools or methods must be highly flexible to meet current as well as far-off needs.
- **The infrastructure is sprawling:** Network environments are becoming more complicated, especially as companies shift their workloads into multiple clouds and hybrid cloud architectures and make the selection of deployment, management, and administration of security solutions more difficult.
- **More stringent requirements for regulatory compliance:** The worldwide regulatory compliance landscape continues to increase by complexity. This makes the compliance of every mandate more challenging.

Data protection tools and platforms

Today, a variety of companies provide data protection platforms and tools. A comprehensive solution should have all of the following features:

- **Discovery:** The ability to discover is often needed to meet regulatory compliance requirements. Look for a tool that can detect and categorize weaknesses across our databases, whether they're hosted in the cloud or on-premises. It will also provide recommendations to address any vulnerabilities that are discovered.
- **Monitoring of Data Activity:** The solution should be capable of monitoring and analysing the entire data activity in all databases, whether our application is on-premises, in the cloud, or inside a container. It will alert us to suspicious activity in real-time to allow us to respond more quickly to threats. It also provides visibility into the state of our information through an integrated and comprehensive user interface. It is also important to choose a system that

enforces rules that govern policies, procedures, and the separation of duties. Be sure that the solution we select is able to generate the reports we need to comply with the regulations.

- **The ability to Tokenize and Encrypt Data:** In case of an incident, encryption is an additional line of protection against any compromise. Any software we choose to use must have the flexibility to protect data cloud, on-premises hybrid, or multi-cloud environments. Find a tool with volume, file, and application encryption features that meet our company's regulations for compliance. This could require tokenization (data concealing) or advanced key management of security keys.
- **Optimization of Data Security and Risk Analysis:** An application that will provide contextual insights through the combination of security data with advanced analytics will allow users to perform optimizing, risk assessment, and reporting in a breeze. Select a tool that is able to keep and combine large amounts of recent and historical data about the security and state of your databases. Also, choose a solution that provides data exploration, auditing, and reporting capabilities via an extensive but user-friendly self-service dashboard.

Best Practices for Database Security

Businesses and organizations heavily rely on databases, which store sensitive and confidential information requiring protection against unauthorized access and data breaches. Implementing robust security measures becomes imperative to ensure the safety of this valuable data and mitigate potential threats effectively.

Here are the data security best practices that will help you to know how to secure your database:



- **Use Strong Passwords**

Passwords serve as the initial barrier against unauthorized entry into databases. Using weak passwords makes it easier for hackers to gain access to sensitive information. It is crucial to use strong passwords that are complex and difficult to guess. A robust password comprises a blend of uppercase and lowercase letters, numbers, and special characters. It is also essential to change passwords frequently, especially when an employee leaves the company. This ensures the previous employee cannot access the database using their old credentials.

Moreover, it is advisable to use two-factor authentication to add an extra layer of security. Two-factor authentication necessitates users to present two types of identification, such as a password combined with either a security token or biometric authentication. This reduces the risk of unauthorized access to databases, even if the password is compromised.

- **Limit Access**

Limiting access to databases is an effective way to prevent unauthorized access to sensitive data. Not all employees or users require access to all the data stored in the database. It is important to restrict access based on the

principle of least privilege, which means granting only the minimum access required to perform a particular task. For example, if an employee requires access to customer data, they should only have access to that section of the database.

Monitoring and tracking user activity within the database is crucial to promptly identifying and reporting unauthorized access or suspicious behavior. By diligently monitoring and auditing user activity, potential security gaps or vulnerabilities within the database can be detected, allowing for timely remediation and fortification of the system.

- **Update and Patch Regularly**

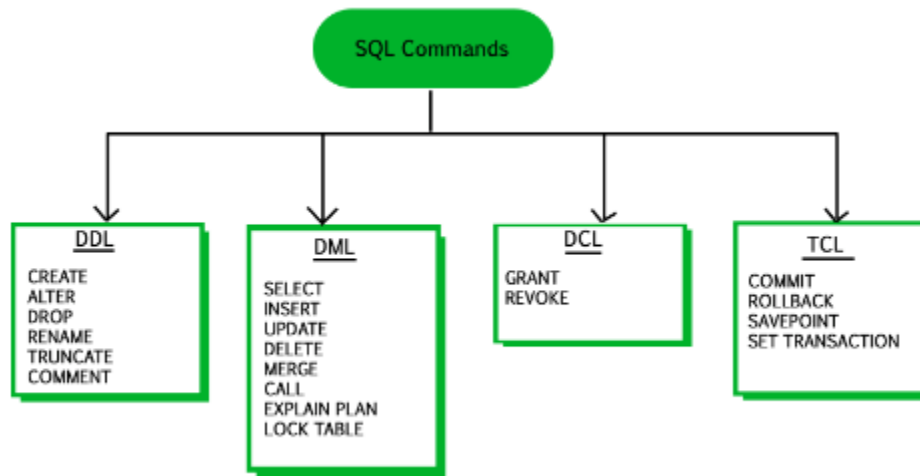
Keeping the database software up-to-date is crucial to preventing any potential security vulnerabilities. Regular updates and patches ensure that any known security vulnerabilities are fixed. It shows that the database is protected against potential threats. Failure to apply updates and patches can result in security breaches that compromise sensitive data.

Maintaining a backup of the database is vital to guaranteeing the ability to restore data in the event of data loss or corruption. It is essential to schedule regular backups and securely store them in a protected location to prevent unauthorized access and ensure data integrity.

- **Monitor for Anomalies**

Detecting and preventing potential security threats can be effectively achieved through the monitoring of database anomalies. Anomaly detection entails the continuous observation of database activities to identify any unusual or abnormal behavior that deviates from the expected norms.

DCL Commands



DCL is an abbreviation for Data Control Language in SQL. It is used to provide different users access to the stored data. It enables the data administrator to grant or revoke the required access to act as the database. When DCL commands are implemented in the database, there is no feature to perform a rollback. The administrator must implement the other DCL command to reverse the action.

- DCL, DDL, DML, DQL, and TCL commands form the SQL (Structured Query Language).
- DCL commands are primarily used to implement access control on the data stored in the database. It is implemented along the DML (Data Manipulation Language) and DDL (Data Definition Language) commands.
- It has a simple syntax and is easiest to implement in a database.
- The administrator can implement DCL commands to add or remove database permissions on a specific user that uses the database when required.

- DCL commands are implemented to grant, revoke and deny permission to retrieve or modify the data in the database.

Types of DCL Commands in SQL

Two types of DCL commands can be used by the user in SQL. These commands are useful, especially when several users access the database. It enables the administrator to manage access control. The two types of DCL commands are as follows:

- GRANT
- REVOKE

GRANT Command

GRANT, as the name itself suggests, provides. This command allows the administrator to provide particular privileges or permissions over a database object, such as a table, view, or procedure. It can provide user access to perform certain database or component operations.

In simple language, the GRANT command allows the user to implement other SQL commands on the database or its objects. The primary function of the GRANT command in SQL is to provide administrators the ability to ensure the security and integrity of the data is maintained in the database.

To have a better understanding of implementing the GRANT statement in the database. Let us use an example.

Implementing GRANT Statement

Consider a scenario where you are the database administrator, and a student table is in the database. Suppose you want a specific user Aman to only SELECT (read)/ retrieve the data from the student table. Then you can use GRANT in the below GRANT statement.

1. **GRANT SELECT ON** student **TO** Aman;

This command will allow Aman to implement the SELECT queries on the student table. This will enable the user to read or retrieve information from the student table.

REVOKE Command

As the name suggests, revoke is to take away. The REVOKE command enables the database administrator to remove the previously provided privileges or permissions from a user over a database or database object, such as a table, view, or procedure. The REVOKE commands prevent the user from accessing or performing a specific operation on an element in the database.

In simple language, the REVOKE command terminates the ability of the user to perform the mentioned SQL command in the REVOKE query on the database or its component. The primary reason for implementing the REVOKE query in the database is to ensure the data's security and integrity.

Let us use an example to better understand how to implement the REVOKE command in SQL.

Implementing REVOKE Command

Consider a scenario where the user is the database administrator. In the above implementation of the GRANT command, the user Aman was provided permission to implement a SELECT query on the student table that allowed Aman to read or retrieve the data from the table. Due to certain circumstances, the administrator wants to revoke the abovementioned permission. To do so, the administrator can implement the below REVOKE statement:

1. **REVOKE SELECT ON** student **FROM** Aman;

This will stop the user Aman from implementing the SELECT query on the student table. The user may be able to implement other queries in the database.

Benefits of Implementing DCL Commands

There are several advantages of implementing Data Control Language commands in a database. Let's see some most common reasons why the user implements DCL commands on the database.

1. **Security:** the primary reason to implement DCL commands in the database is to manage the access to the database and its object between different users. This limits the actions that can be performed by specific users on the different elements in the database. It ensures the security and integrity of the data stored in the database.
2. **Granular control:** DCL commands provide granular control to the data administrator over the database. It allows the administrator to provide or remove specific privileges or permissions from other users using a database for information. Thus, it enables the admin to create different levels of access to the database.

3. **Flexibility:** The data administrator can implement DCL commands on specific commands and queries in the database. It allows the administrator to grant or revoke user permissions and privileges as per their needs. It provides flexibility to the administrator that allows them to manage access to the database.

Disadvantages of Implementing DCL Commands

Along with the benefits of implementing DCL commands in the database, they have some disadvantages. Some of the common disadvantages of implementing DCL commands are as follows:

1. **Complexity:** It increases the complexity of database management. If many users are accessing the database, keeping track of permission and privileges provided to every user in the database becomes very complex.
2. **Time-Consuming:** In most organizations, several users access the database, and different users have different access levels to organization data. It is time-consuming to assign the permissions and privileges to each user separately.
3. **Risk of human error:** Human administrators execute DCL commands and can make mistakes in granting or revoking privileges. Thus, giving unauthorized access to data or imposing unintended restrictions on access.
4. **Lack of audit trail:** There may be no built-in mechanism to track changes to privileges and permissions over time. Thus, it is extremely difficult to determine who has access to the data and when that access was granted or revoked.

